Complete SQL Learning Order

Tiered Roadmap for App Development and Interview Readiness (SQL Server / T-SQL)

This is a complete, dependency-ordered topic list for SQL, covering true fundamentals through advanced T-SQL programmability, organized into three tiers by real relevance to both application development and interviews. It is built for someone with prior data lake migration and stored procedure experience, so T-SQL programmability gets full depth rather than being treated as a minor topic. Tier 1 topics should be mastered deeply. Tier 2 topics should be understood solidly and reached for when relevant. Tier 3 topics require only awareness. A final, separate section covers GCP/BigQuery concepts at a light, conceptual level for migration-related interview discussions.

How to Practice: One Running Schema

Use one consistent schema throughout, instead of disconnected practice each time: a Retail Order Database with tables for Customers, Products, Categories, Orders, OrderItems, and Employees (who process orders). Build this schema as you reach Section 2 (Database Design), then every later "Practice" box tells you what to query, modify, or build against this same schema. Where a practice box references an external platform (e.g. LeetCode/HackerRank SQL problems), do that in addition — the platform problems train pattern recognition on unfamiliar schemas, which interviews require.

TIER 1 — MUST MASTER

Core querying, schema design, and T-SQL programmability. Build deep fluency here — this covers both daily app-development querying and the bulk of real interview questions, including stored-procedure-specific ones tied to your migration background.

0. Foundations — What a Relational Database Is

0.1. Tables, rows, columns — the relational model

0.2. What a database engine does (SQL Server specifically, conceptually)

0.3. SQL sub-languages — DDL, DML, DCL, TCL (what each category means)

0.4. Schemas and databases — how SQL Server organizes objects

0.5. SQL Server Management Studio (SSMS) or Azure Data Studio — basic orientation

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Install SQL Server Express (or use a free Azure SQL/local Docker instance) and SSMS or Azure Data Studio. Create an empty database called RetailOrderDb — no tables yet, just confirm you can connect and run a query.

1. Core Querying — SELECT Fundamentals

1.1. SELECT, FROM, column aliasing (AS)

1.2. WHERE — filtering with comparison operators

1.3. Logical operators — AND, OR, NOT

1.4. NULL handling — IS NULL, IS NOT NULL, and why NULL = NULL is never true

1.5. ORDER BY — sorting, ASC/DESC, multiple columns

1.6. DISTINCT

1.7. TOP / OFFSET-FETCH — limiting result sets and pagination

1.8. LIKE and wildcards — pattern matching

1.9. IN, BETWEEN

1.10. CASE expressions — conditional logic inside a query

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Practice: SQLZoo or HackerRank "Basic Select" section problems. Then create your RetailOrderDb Products table by hand (manually insert 8-10 rows) and write 8 different SELECT queries against it covering filtering, sorting, pagination, and CASE-based labeling (e.g. "label products as Cheap/Mid/Expensive by price").

2. Database Design Fundamentals

2.1. Primary keys — purpose and constraints

2.2. Foreign keys — enforcing relationships

2.3. Data types in SQL Server — numeric, string (VARCHAR vs NVARCHAR vs CHAR), date/time, BIT

2.4. NOT NULL, DEFAULT, CHECK, UNIQUE constraints

2.5. One-to-many, many-to-many relationships and junction tables

2.6. Normalization — 1NF, 2NF, 3NF (concept and why it matters)

2.7. When and why to denormalize (practical tradeoffs, not just theory)

2.8. CREATE TABLE, ALTER TABLE, DROP TABLE

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

HIGH RELEVANCE FOR YOUR BACKGROUND — schema design questions often come up alongside migration discussions ("why was the schema structured this way"). Build the full RetailOrderDb schema: Customers, Products, Categories (one-to-many with Products), Orders, OrderItems (junction table between Orders and Products, many-to-many), and Employees. Add appropriate primary keys, foreign keys, and at least one CHECK constraint (e.g. price >= 0).

3. Joins

3.1. INNER JOIN

3.2. LEFT JOIN / RIGHT JOIN

3.3. FULL OUTER JOIN

3.4. CROSS JOIN

3.5. Self joins

3.6. Joining more than two tables

3.7. Common join mistakes (accidental cartesian products, wrong join direction for LEFT/RIGHT)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

HIGH IMPORTANCE — joins are the single most-tested SQL skill in both interviews and app development. Practice: HackerRank SQL "Join" section, LeetCode #175 (Combine Two Tables), LeetCode #181 (Employees Earning More Than Their Managers, self join). Then: write a query joining Orders, OrderItems, Products, and Customers to produce "customer name, product name, quantity, line total" for every order in RetailOrderDb.

4. Aggregation and Grouping

4.1. Aggregate functions — COUNT, SUM, AVG, MIN, MAX

4.2. GROUP BY

4.3. HAVING vs WHERE — when each applies

4.4. Grouping by multiple columns

4.5. Combining aggregation with joins

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

HIGH IMPORTANCE — aggregation questions are extremely common ("total sales per customer", "top N products by revenue"). Practice: LeetCode #182 (Duplicate Emails), LeetCode #586 (Customer Placing the Largest Number of Orders). Then: write "total revenue per customer", "average order value per month", and "top 5 best-selling products" queries against RetailOrderDb.

5. Subqueries

5.1. Subqueries in WHERE (single-value and IN-based)

5.2. Subqueries in SELECT (scalar subqueries)

5.3. Subqueries in FROM (derived tables)

5.4. Correlated subqueries

5.5. EXISTS / NOT EXISTS vs IN / NOT IN — differences and when to prefer each

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Practice: LeetCode #1084 (Sales Analysis III), LeetCode #1350 (Students With Invalid Departments, NOT EXISTS pattern). Then: rewrite at least 2 of your join-based queries from Section 3-4 as correlated subqueries instead, to feel the tradeoff in readability and performance.

6. Common Table Expressions (CTEs) and Recursive Queries

6.1. WITH clause — basic CTEs for readability

6.2. Multiple/chained CTEs

6.3. Recursive CTEs — mechanics (anchor + recursive member)

6.4. Recursive CTEs for hierarchical data (e.g. org charts, category trees)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Often forgotten in roadmaps but frequently interview-tested. Practice: LeetCode #1757 (Recyclable and Low Fat Products, basic CTE practice), then write a recursive CTE to traverse an Employee table with a ManagerId self-reference, producing a full reporting hierarchy.

7. Set Operations

7.1. UNION vs UNION ALL

7.2. INTERSECT

7.3. EXCEPT

7.4. Rules for combining result sets (matching column counts/types)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Practice: combine a "customers who ordered in 2024" query and a "customers who ordered in 2025" query using UNION, INTERSECT, and EXCEPT to find new, returning, and lapsed customers respectively.

8. Window Functions

8.1. OVER() clause — the core concept of window functions

8.2. PARTITION BY vs GROUP BY — the key distinction

8.3. ROW_NUMBER(), RANK(), DENSE_RANK()

8.4. LAG() and LEAD()

8.5. Running totals and moving averages (SUM/AVG OVER with ORDER BY)

8.6. NTILE() for bucketing

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

HIGH IMPORTANCE — window functions are a top interview differentiator and extremely common in real reporting queries (which connects directly to data lake/analytics work). Practice: LeetCode #176 (Second Highest Salary, can be solved with window functions), LeetCode #185 (Department Top Three Salaries), LeetCode #1321 (Restaurant Growth, running average). Then: write a query ranking products by revenue within each category using RANK(), and a running total of daily order revenue using SUM() OVER.

9. Data Modification — INSERT, UPDATE, DELETE, MERGE

9.1. INSERT — single row, multiple rows, INSERT...SELECT

9.2. UPDATE — with WHERE, updating based on a join

9.3. DELETE — with WHERE, and the danger of omitting it

9.4. TRUNCATE vs DELETE — differences

9.5. MERGE statement — upsert logic (insert/update/delete in one statement)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

RELEVANT TO MIGRATION WORK — MERGE is heavily used in ETL/data sync scenarios, directly relevant to your data lake migration experience. Practice: write an UPDATE that applies a 10% discount to all products in a given category using a join-based WHERE, then write a MERGE statement that syncs a staging table of new/changed products into your main Products table.

10. Transactions and Concurrency

10.1. BEGIN TRANSACTION, COMMIT, ROLLBACK

10.2. ACID properties — what each guarantees, in practical terms

10.3. Isolation levels (READ COMMITTED, READ UNCOMMITTED, SERIALIZABLE — concept-level)

10.4. Deadlocks — what causes them, how to recognize one

10.5. Locking basics (concept-level: shared vs exclusive locks)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Practice: wrap an order-placement sequence (insert into Orders, insert into OrderItems, update Products stock) in an explicit transaction, and deliberately test what happens if you ROLLBACK partway through versus COMMIT.

11. T-SQL Programmability — Variables and Control Flow

11.1. DECLARE, variable assignment, basic data types in variables

11.2. IF...ELSE

11.3. WHILE loops

11.4. Batches and GO separator (concept)

11.5. PRINT and basic debugging output

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

HIGH RELEVANCE FOR YOUR BACKGROUND — this is the procedural foundation everything in stored procedures builds on. Practice: write a T-SQL script using a WHILE loop and IF logic to generate 100 sample test orders with randomized customers/products (a realistic "seed data" task you likely did in your migration work).

12. Stored Procedures

12.1. CREATE PROCEDURE — syntax and structure

12.2. Input parameters and default values

12.3. Output parameters

12.4. Returning result sets vs return codes

12.5. EXEC / EXECUTE — calling a stored procedure

12.6. ALTER PROCEDURE, DROP PROCEDURE

12.7. Why stored procedures are used in real systems — performance, security, encapsulation, reducing app-to-DB round trips

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

HIGHEST RELEVANCE TO YOUR INTERVIEW STORY — you have direct migration experience here; expect interviewers to ask you to explain or write one. Practice: write a stored procedure GetCustomerOrderHistory that takes a CustomerId parameter and returns their full order history with product details, then write one that takes order details as parameters and performs the full "place an order" sequence (insert order, insert items, update stock) as a single transactional procedure.

13. Functions (User-Defined Functions)

13.1. Scalar functions — CREATE FUNCTION returning a single value

13.2. Table-valued functions (inline and multi-statement)

13.3. Built-in functions you should know cold — string functions (SUBSTRING, CONCAT, TRIM, LEN), date functions (DATEADD, DATEDIFF, GETDATE), conversion functions (CAST, CONVERT, TRY_CONVERT)

13.4. Differences between stored procedures and functions — when to use which

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Practice: write a scalar function CalculateDiscountedPrice(price, discountPercent), and a table-valued function GetOrdersByDateRange(startDate, endDate) that you can call directly inside a SELECT's FROM clause.

14. Error Handling in T-SQL

14.1. TRY...CATCH blocks

14.2. ERROR_MESSAGE(), ERROR_NUMBER(), ERROR_LINE() and related functions

14.3. RAISERROR and THROW — raising custom errors

14.4. Combining TRY...CATCH with transactions (rollback on error)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

RELEVANT TO MIGRATION WORK — robust error handling in procedures is exactly the kind of detail that comes up when discussing why a migrated procedure needed rework. Practice: wrap your "place an order" stored procedure from Section 12 in TRY...CATCH, rolling back the transaction and re-throwing a clear custom error if stock is insufficient.

15. Triggers

15.1. AFTER triggers — INSERT, UPDATE, DELETE triggers

15.2. INSTEAD OF triggers (concept)

15.3. INSERTED and DELETED virtual tables — how triggers access changed data

15.4. When triggers are appropriate vs when they cause maintenance headaches (practical judgment, not just syntax)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Practice: write an AFTER UPDATE trigger on Products that logs any price change into a ProductPriceHistory audit table, capturing old and new price using the INSERTED/DELETED tables.

16. Views

16.1. CREATE VIEW — purpose and basic syntax

16.2. Updatable views (concept and limitations)

16.3. Indexed views (concept-level awareness)

16.4. When a view is the right tool vs a stored procedure or function

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

Practice: create a view CustomerOrderSummary that exposes total orders and total spend per customer, simplifying what would otherwise be a repeated join+aggregation query.

17. Indexing and Query Performance Basics

17.1. Clustered vs non-clustered indexes — what each actually does

17.2. How an index speeds up a WHERE/JOIN — conceptual model (not internals-deep)

17.3. Reading a basic execution plan — identifying a seek vs a scan

17.4. Common performance anti-patterns — SELECT *, functions on indexed columns in WHERE, missing indexes on foreign keys

17.5. Composite indexes and column order (concept)

PRACTICE — RETAIL ORDER DB / PLATFORM PROBLEMS

HIGH RELEVANCE FOR YOUR BACKGROUND — performance discussion is a near-guaranteed interview topic for anyone with migration/ETL experience. Practice: run a query against RetailOrderDb filtering Orders by CustomerId without an index, view its execution plan (look for a Table Scan), add a non-clustered index on CustomerId, re-run, and confirm the plan now shows an Index Seek.

TIER 2 — SHOULD KNOW

Solid, real-world topics that come up periodically in both app development and interviews, but don't need Tier 1's depth of repetition. Understand these well enough to use them correctly when the situation calls for it.

- Date and time handling depth — DATEPART, time zones (DATETIMEOFFSET), formatting dates for output
- PIVOT and UNPIVOT — reshaping rows into columns and back
- String aggregation — STRING_AGG for combining rows into a delimited string
- Temporary tables vs table variables (#Temp vs @TableVariable) — differences and when to use which
- Dynamic SQL — building and executing SQL strings (and the SQL injection risk this introduces)

- Cursors — concept and basic syntax (know it exists and why it's usually avoided in favor of set-based operations)
- Bulk operations — BULK INSERT, OPENROWSET (concept-level, relevant to data migration contexts)
- Linked servers (concept-level awareness, relevant to cross-database migration work)
- Basic security — GRANT/REVOKE/DENY permissions, the concept of least-privilege access
- Schema-bound objects and naming conventions in larger codebases
- JSON support in SQL Server — OPENJSON, FOR JSON (relevant given modern app/API data exchange)

TIER 3 — AWARE OF ONLY

Specialized or administrative topics that rarely come up in app-development or standard interview contexts unless you move toward a DBA-focused role. Know the term, not the implementation.

Database backup and restore strategies
Full/differential/log backups, recovery models. DBA-territory; relevant only if asked about disaster recovery conceptually.

Replication
Mechanisms for copying data between SQL Server instances. Rarely relevant outside infrastructure-focused roles.

Partitioning (SQL Server table partitioning)
Splitting large tables across storage for performance at scale. Worth knowing the term exists, especially given your data lake background, but implementation is a specialized skill.

Always On Availability Groups / clustering
High-availability infrastructure topics, DBA/infrastructure-team territory.

Query Store and advanced performance tuning
Deep performance monitoring tooling; awareness only unless pursuing a performance-tuning-focused role.

CLR integration (running .NET code inside SQL Server)
Rare, legacy-leaning feature. Know it exists, essentially never used in modern app development.

Service Broker
SQL Server's internal messaging/queueing feature. Largely superseded by external message queues in modern architectures.

GCP / BigQuery Bridge — For Your Migration Background

This section is intentionally light and conceptual, not a deep BigQuery course. The goal is to be able to speak fluently about your SQL-Server-to-GCP migration experience in interviews, and to recognize the key differences if asked to write or read BigQuery SQL.

- BigQuery's architecture at a glance — columnar, distributed, serverless query execution (vs SQL Server's row-based, instance-based model)
- Standard SQL in BigQuery vs T-SQL — key syntax differences you'd actually hit (backticks for table names, STRUCT and ARRAY types, no stored procedures in the traditional sense historically, though scripting now exists)
- Partitioning and clustering in BigQuery — conceptual equivalent to indexing, but designed for huge scan-based workloads rather than seek-based lookups

- Cost model awareness — BigQuery charges based on data scanned, which directly affects how queries should be written (avoiding SELECT *, partition pruning)
- Schema-on-read vs schema-on-write — the core data lake concept that likely came up in your migration work
- Batch ETL concepts relevant to migration — staging tables, data validation steps, the general shape of a SQL Server-to-GCP migration pipeline (conceptual, not tool-specific)
- Where stored procedure logic typically goes when migrating away from T-SQL — rewritten as BigQuery scripting, dbt models, or moved into an application/orchestration layer (this is exactly the kind of judgment call interviewers will ask you to narrate from experience)

Note on ordering: Tier 1 sections are sequenced so each assumes the previous — joins before aggregation, aggregation before subqueries, subqueries before CTEs, and all core querying before T-SQL programmability (Sections 11-16), since procedures/functions/triggers are written using everything before them. Indexing and performance (Section 17) is placed last in Tier 1 deliberately: reasoning about performance only becomes meaningful once you've written enough real queries to have something to optimize. The GCP/BigQuery section is fully separated at the end and is not a prerequisite for anything in Tiers 1-3; it exists solely to support discussing your specific migration project confidently in interviews.